

SIMATSENGINEERING

ASSESSMENTTOOL2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1703

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 5

Total Marks:50

Annexure A **Scenario**

Based Assignment

Code of Conduct:

*I **B.Theerthi [192473026]** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: B.Theerthi

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

AI-Powered Drone for Emergency Medicine Delivery

i. Greedy Best-First Search (GBFS)

Greedy Best-First Search selects the next path based only on the heuristic value $h(n)$ (estimated distance to the destination).

Evaluation Function:

$$f(n)=h(n)$$

How it works

- The drone always chooses the path that appears closest to the destination.
- It ignores the distance already traveled and movement costs. □ Uses straight-line distance as the heuristic.

Strengths

Very fast.

Requires less computation.

-
-
-

- Suitable for emergency situations requiring quick decisions.
- Uses less memory than many search algorithms.

Weaknesses under uncertainty

- May choose risky or blocked paths.
- Does not consider weather or terrain costs.
- Can produce non-optimal routes.
- Needs repeated replanning when roads become blocked.

ii. A* Search

A* considers both:

- Actual travel cost $g(n)$
- Estimated remaining cost $h(n)$ **Evaluation Function**

$$f(n)=g(n)+h(n) \quad f(n)=g(n)+h(n) \quad f(n)=g(n)+h(n)$$

How it works

The drone:

- Calculates total cost of every possible route. □ Includes:
 - fuel consumption
 - weather conditions
 - terrain difficulty
 - blocked roads
- Chooses the path with minimum total estimated cost.

Advantages

- Produces optimal route when heuristic is admissible.
- Safer than Greedy Search.
- Adapts better to changing travel costs. □ Balances speed and accuracy.

Disadvantages

More computation.
Requires more memory.
Slightly slower.

iii. Impact of Heuristic Accuracy

-
-
-

Accurate Heuristic

- Both algorithms reach destination faster.
- Fewer nodes explored. □ Better path selection.

Poor Heuristic

Greedy Search:

- Performance drops significantly.
- Can choose wrong paths.
- May get trapped near blocked roads.

A*:

- Still finds optimal solution.
- Only becomes slower because more nodes are explored.

Therefore, A* is less sensitive to heuristic errors.

iv. Effect of Dynamic Changes

Example:

- Flood suddenly blocks a bridge.

Greedy Search

- Continues toward blocked road.
- Must restart search. □ Wastes time.

A*

- Updates path cost.
- Recalculates best route.
- More reliable in dynamic environments.

Dynamic obstacles increase:

computation
replanning search
complexity

□
□
□

v. Recommended Algorithm

Recommended: A*

Justification

Criteria	Greedy	A*
Speed	High	Moderate
Optimality	No	Yes
Safety	Medium	High
Dynamic updates	Poor	Good
Considers travel cost	No	Yes

Conclusion

A* is the best choice because it:

- finds safer routes
- minimizes delay
- considers weather and terrain □ avoids dangerous paths □ provides optimal navigation.

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- Explain how Hill Climbing would work in this scenario and identify its limitations
- Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- Recommend the best approach and justify your choice based on scalability and adaptability

Smart Traffic Signal Optimization

Optimization Problem

Objective

Minimize

- Total waiting time
- Fuel consumption
- Traffic congestion

Decision Variables

- Green signal duration
- Signal sequence
- Phase timing

Constraints

- Minimum green time
- Maximum waiting time
- Pedestrian crossing
- Emergency vehicle priority

Why Traditional Search is Inefficient

Traditional search explores every possible signal combination.

For hundreds of intersections,

Search space becomes enormous.

Problems:

- Huge branching factor
- Slow computation
- Cannot respond in real time □ Memory intensive

i. Hill Climbing

Hill Climbing starts with one solution.

It repeatedly changes signal timings.

If the new timing improves traffic flow,
move to the new solution.

Otherwise stop.

Advantages

- Very fast.
- Easy implementation. □ Low memory.

Limitations

- Gets stuck in local optimum.
- Cannot guarantee global optimum. □ Sensitive to starting

solution. **ii. Local Maxima, Plateaus and Ridges**

Local Maximum

A solution better than nearby solutions but not the best overall.

Example:

One junction improves while the entire city still has congestion.

Plateau

Many neighboring solutions have equal quality.

Example:

Changing signal timing by 2 seconds gives no improvement.

Algorithm cannot decide where to move.

Ridge

Improvement requires several coordinated changes.

Example:

Changing only one signal does not help.

Changing five nearby intersections together reduces congestion.

Hill climbing struggles here. **iii. Simulated**

Annealing and Genetic Algorithm

Simulated Annealing

Allows occasional bad moves.

Benefits:

- Escapes local maxima.

- Better exploration.
- Eventually converges to good solution.

Genetic Algorithm

Works like biological evolution.

Steps:

1. Generate many signal plans.
2. Evaluate fitness.
3. Select best plans.
4. Perform crossover.
5. Apply mutation.
6. Repeat.

Advantages:

- Handles huge search spaces.
- Avoids local optima.
- Supports parallel optimization.

iv. Recommended Approach

Best Choice: Genetic Algorithm

Justification

- Excellent scalability.
- Handles many intersections.
- Adapts to changing traffic.
- Produces near-optimal solutions.
- Works well for large optimization problems.

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search**

ii. **Analyze the challenges of partial observability and dynamic environments** iii.

Describe how the rover builds and updates its internal model iv. **Compare online search with uninformed and informed search strategies**

v. **Justify the most suitable approach considering uncertainty, adaptability, and safety**

Autonomous Mars Rover

i. Why Online Search?

Offline search requires a complete map.

Mars rover has:

- Unknown terrain
- Limited sensors
- Hidden obstacles

Therefore,

It must:

Observe → Decide → Move → Learn

This is online search.

ii. Challenges

Partial Observability

The rover cannot see the whole environment.

Hidden hazards include:

- Rocks
- Craters
- Sand pits

Dynamic Environment

Dust storms, Loose soil, New obstacles, Sensor uncertainty

The environment changes continuously.

iii. Internal Model

The rover stores:

- Explored locations
- Hazard locations
- Safe paths
- Sample locations
- Battery status Whenever new information arrives, its internal map is updated.

Future decisions become better.

iv. Comparison

Feature	Uninformed	Informed	Online
Complete map needed	Yes	Yes	No
Uses heuristic	No	Yes	Optional
Learns while moving	No	No	Yes
Handles uncertainty	Poor	Moderate	Excellent

v. Recommended Approach

Online Search Agent

Reasons:

- Learns continuously.
- Works without complete map.
- Avoids unexpected hazards.
- Adapts to new information. □ Safer for Mars exploration.

4. Scenario: A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- Clearly define variables, domains, and constraints with explanation**
- Explain how backtracking search works in this scenario**
- Discuss how constraint propagation (e.g., forward checking) improves efficiency**
- Analyze real-world challenges and justify the best strategy for scalability**

University Exam Timetable (CSP)

CSP Formulation

Variables

Each course represents one variable.

Example:

Math

Physics

AI

DBMS

Domains

Possible exam slots

Example:

Monday Morning

Monday Afternoon

Tuesday Morning

Tuesday Afternoon

Constraints

1. No overlapping exams for same student.
 2. Hall capacity must not exceed limit.
 3. Faculty availability.
 4. Subject ordering.
 5. Special accommodation.
- ## ii. Backtracking Search

Steps

1. Assign first course.
2. Check constraints.
3. Assign next course.
4. If conflict occurs, undo previous assignment.
5. Try another slot.
6. Continue until complete timetable.

Example

AI → Monday Morning

DBMS → Monday Morning

Conflict

Backtrack

Assign DBMS → Tuesday Morning

Continue.

iii. Constraint Propagation (Forward Checking)

Whenever one slot is assigned,

Remove conflicting slots from remaining subjects.

Example

Math assigned Monday Morning.

Physics shares students.

Remove Monday Morning from Physics.

Benefits

- Detects conflicts early.
 - Reduces search space.
 - Less backtracking. □ Faster scheduling.
- #### **iv. Real-world Challenges**
- Thousands of students.
 - Hundreds of courses.
 - Limited halls.
 - Last-minute faculty changes. □ Special accommodations.

Best Strategy

Backtracking + Forward Checking + Constraint Propagation

Advantages

- Efficient.
- Scalable.
- Produces valid timetable.
- Reduces unnecessary computation.

5. Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & AlphaBeta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

Strategic Game AI (Minimax & Alpha-Beta Pruning)

i. Minimax Algorithm

Game is represented as a decision tree.

Levels alternate:

MAX → AI player

MIN → Human player

AI assumes the opponent always plays the best move.

The AI chooses the move that maximizes its minimum guaranteed score.

ii. Computational Challenges

Problems

- Huge branching factor.
- Millions of game states. □ High memory usage.
- Long computation time.

Time Complexity

$O(b^d)$

where b = branching

factor d = search depth

iii. Alpha-Beta

Pruning

Alpha-Beta pruning removes branches that cannot improve the final decision.

Alpha (α)

Best value found for MAX.

Beta (β)

Best value found for MIN.

If

$$\alpha \geq \beta$$

remaining branches are ignored.

Benefits

- Fewer nodes explored.
- Same optimal decision as Minimax.
- Faster execution.
- Enables deeper search within the same time.

iv. Evaluation Function

The AI assigns a score to every game state.

Example factors:

- Number of soldiers.
- Available resources. □ Territory controlled.
- Health of units.
- Defensive strength.
- Attack opportunities.
- Distance to enemy base.

Example

$$\text{Score} = (5 \times \text{Resources}) + (10 \times \text{Army}) + (15 \times \text{Territory}) - (8 \times \text{Enemy Strength})$$

Higher score indicates a stronger position.

v. Recommended Strategy

Best Choice

Minimax + Alpha-Beta Pruning + Evaluation Function

Justification

- Produces optimal decisions.
- Reduces unnecessary computation.
- Meets real-time constraints.
- Handles complex game states efficiently.
- Widely used in game AI because it balances decision quality with computational efficiency.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts

Decision Making	20%	Makes wellreasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand